


# Build night 4

## Backtest Validation and Failure Testing

Try to break your own backtest before the market does.

Travis Sim

NUS · Singapore

Thu 28 Jan 2027 · COM3/COM4

# Where we are

Tonight we try to break our own backtest, then break our own agent.

DONE

## Workshop 3

Thu 21 Jan

Deploy a venue component on devnet

TONIGHT

## Build night 4

Thu 28 Jan

Backtest validation and failure testing

NEXT

## Workshop 4

Thu 4 Feb

Performance scorecard and shipping

# Tonight

18:00 to 20:00, mostly hands on.

**18:00** — 10 min
### Progress circle
last result and tonight's question

**18:10** — 10 min
### Mini-talk
leakage and the best-of-many-tries trap

**18:20** — 20 min
### Demo tour
you predict, then we run it

**18:40** — 20 min
### Guided mission
find the planted leak, fix it

**19:00** — 50 min
### Free build
your own project

**19:50** — 10 min
### Show and tell
push, one line on next step

# Three ways to fool yourself

A backtest can look great and mean nothing.

1 LEAKAGE

## 60.4% to 53.9%

Balanced accuracy of the same memoriser: shuffled folds, then time folds plus purge and embargo (label horizon 24). A coin flip is 50%.

2 BEST OF MANY TRIES

## 1.00 to 0.17

Score of the best of 50 random strategies: on the data it was picked on, then on data it never saw.

3 A BROKEN FEED

## 95 s

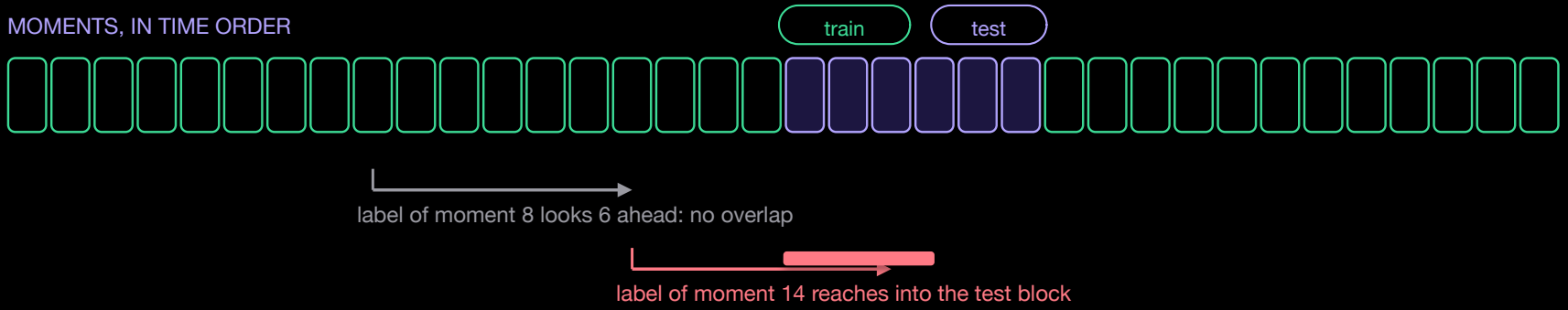
Time for the alarm to catch a simulated oracle freeze. No false alarm on the healthy 40 minutes.

Numbers from demo-repo/code/out (splits.json, overfit.json, breaks.json). Simulated, on recorded data; not advice.

# Overlapping labels leak

A label looks ahead, so neighbours share answers.

Every moment is labelled by what happens in the next h samples. Two moments next to each other share most of that future.



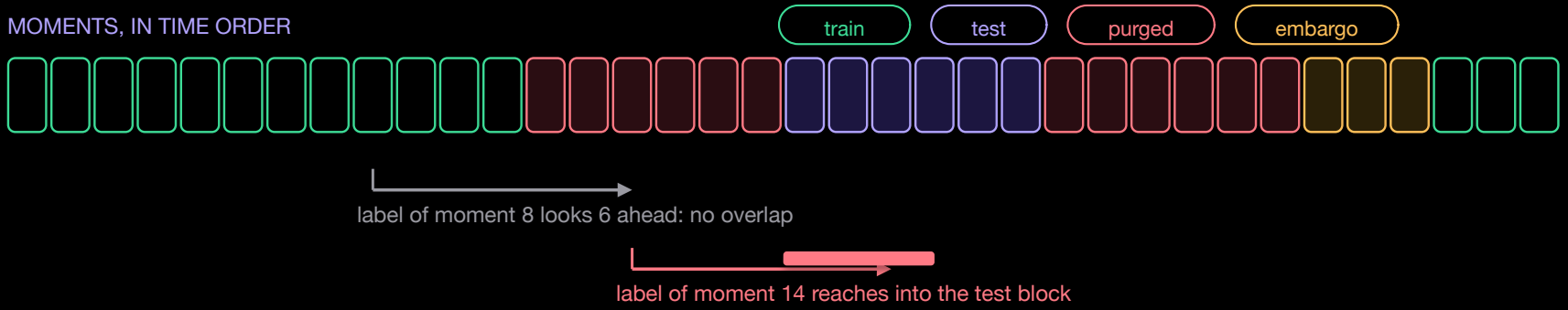

A training moment right before the test block already knows part of the test answer.

Illustrative; h = 6 here. The real data uses h from 6 to 48 samples of 5 seconds.

# Purge and embargo

Remove the neighbours that know too much.

Purge: drop training moments whose label overlaps a test label. Embargo: drop a few more after the test block.




Fewer training moments, an honest score.

Illustrative; h = 6, embargo = 3 here.

# The planted leak

One off-by-one, and today's decision sees tomorrow's price.

src/leaky.ts

```
export function trendSignal(px: number[], i: number, o: TrendOpts = {}): 1 | -1 | 0 {
  const fast = o.fast ?? 2, slow = o.slow ?? 10;
  const shift = o.leak ? 1 : 0;                                  <- the planted bug
  if (i - slow + 1 < 0) return 0;
  const end = i + 1 + shift; // slice end is exclusive
  const f = mean(px.slice(end - fast, end));
  const s = mean(px.slice(end - slow, end));
  return f > s ? 1 : -1;
}
```

THE CAUSALITY TEST

```
if (signal(px, i) !== signal(px.slice(0, i + 1), i)) { leaking++; flagged.push(i); if
(first === null) first = i; }
```

PROFIT, BEFORE COSTS

| leaky | fixed |
|---|---|
| +18.84 USD | -1.76 USD |

CAUSALITY TEST: MOMENTS THAT CHANGE

| leaky | fixed |
|---|---|
| 37 of 440 | 0 of 440 |

61 trades each. Simulated, on recorded data; not advice.

Source: demo-repo/code/src/leaky.ts (excerpt: comments and blank lines removed). Simulated, on recorded data; not advice.

# Best of many tries

## 50 random strategies, one winner in-sample.




WINNER IN-SAMPLE

**1.00**

coin-flip #704396 hold18, 8 trades

SAME STRATEGY ON FRESH DATA

**0.17**

5 trades; rank 30 of 48

The winner is picked on the left axis. The fresh data is the bottom axis.

Source: code/out/overfit.json, draw 1, before costs. Simulated, on recorded data; not advice.

# The luck benchmark

The more you try, the better the best looks, by luck alone.




20 DRAWS OF 50, BEFORE COSTS

**0.51 to 0.12**

average best score in-sample, then the same strategies on fresh data

WINNER BEATS ITS BENCHMARK

**4 of 20**

draws; a score only counts if it clears the luck line

WITH 5 + 2 BPS COSTS PER LEG

**-1.39**

average best score in-sample; positive on fresh data in 0 of 20

Simulated, on recorded data; not advice.

Source: code/out/overfit.json (curveSeed1 and summary), before costs. Simulated, on recorded data; not advice.

# Real numbers

## 40 minutes of real SOL/USDC prices from Jupiter, Pyth and Orca.

SAMPLES

481

one every 5 seconds

WINDOW

40 min

01:13 to 01:53 Singapore time, 21 Oct 2026 (UTC evening)

TEST MOMENTS

433

5 folds; one standard error is about 2.4 points

STRATEGIES

20 x 50

20 draws of 50 random strategies

MEMORISER BALANCED ACCURACY (coin flip = 50%)



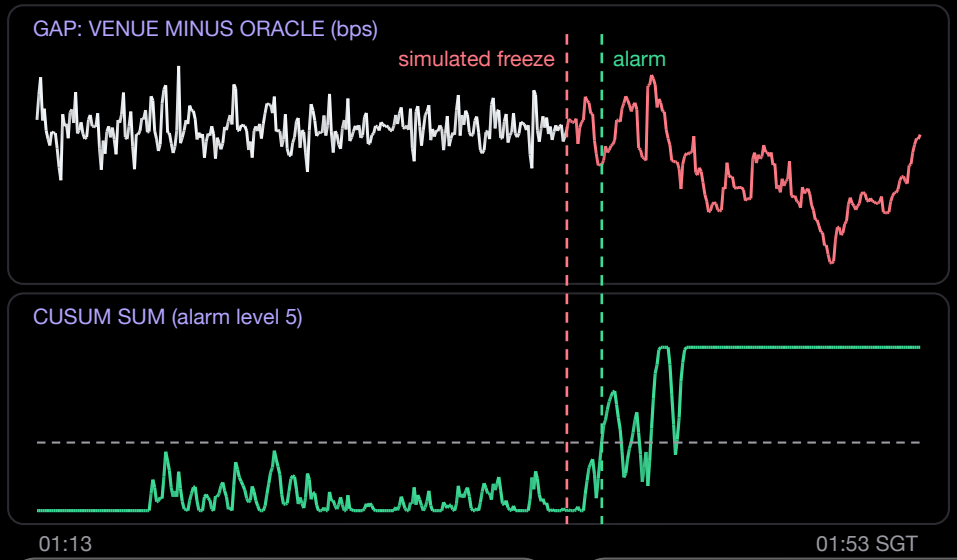

Balanced accuracy: guessing one side every time scores 50%. This window drifted up, so plain accuracy of always guessing up would be 63.5% at h = 24. Read the gap between fold schemes, not the level.

Reads use public mainnet data; every write is devnet only; no real funds.

Source: code/out/splits.json, overfit.json and recorded/SOURCES.md; recorded data from earlier build nights, last sample 2026-10-21 17:53:59 UTC. Public mainnet reads. Simulated, on recorded data; not advice.

# A broken feed raises an alarm

Watch the gap between the venue price and the oracle.







`Try it: https://nus-solana-demos.vercel.app/0007/04-feed-break.html`

Live mode: switch it on to watch real Solana data.

**BASELINE GAP**

2.0 bps — sd 2.7 bps

**HEALTHY FEED**

no alarm — across all 40 minutes

**SIMULATED FREEZE**

95 s — from freeze to alarm

Source: code/out/breaks.json. The freeze is SIMULATED; the venue price and the gap before it are real.

# Five Solana failures

We inject each one on top of real prices. Nothing live is attacked.

### 1

**Stale oracle**

The oracle stops updating while the market moves 80 bps.

### 2

**RPC outage**

HTTP 429 for 5 minutes. The agent keeps its last prices.

### 3

**Slippage shock**

Every fill is 40 bps worse than expected.

### 4

**Fee change**

Fee per leg jumps from 5 to 30 bps.

### 5

**Duplicate submission**

A timeout makes the agent send every order twice.

A paper agent sends one 250 USD order every 30 seconds through the build night 1 risk hook (limits 1000 USD, 50 bps from the oracle, oracle no older than 60 s). The fault lasts 5 minutes from 01:37 Singapore time.

Source: code/src/faults.ts. Simulated, on recorded data; not advice.

# What the hook caught and missed

The build night 1 risk hook, unchanged, against each fault.

| FAULT | ORDERS | THE HOOK | LOSS, HOOK | LOSS, NO HOOK | WITH EXTRA GUARD |
|---|---|---|---|---|---|
| Stale oracle | 10 | CAUGHT 8 of 10 | $2.02 | $10.04 | no extra guard needed |
| RPC outage (429) | 10 | CAUGHT 8 of 10 | $2.05 | $10.17 | no extra guard needed |
| Slippage shock | 10 | MISSED ALL | $10.00 | $10.00 | breaker: 3 accepted, $3.00 |
| Fee change | 10 | MISSED ALL | $6.25 | $6.25 | none yet: $6.25 extra fees |
| Duplicate submission | 20 | MISSED ALL | $5,000 exposed | $5,000 exposed | idempotency key: 10 accepted |

Re-priced strategies (net, whole 40 min): trend -89.4 USD at 40 bps slippage and -63.4 USD at a 30 bps fee; basis -169.7 and -120.3. HTTP 429 for 300 s: 300 calls with a 1 s retry, 14 with back-off.

Source: code/out/faults.json. 10 orders of 250 USD in the 5-minute fault window (20 for duplicates). Simulated, on recorded data; not advice.

# Demo: split leakage

PREDICT FIRST

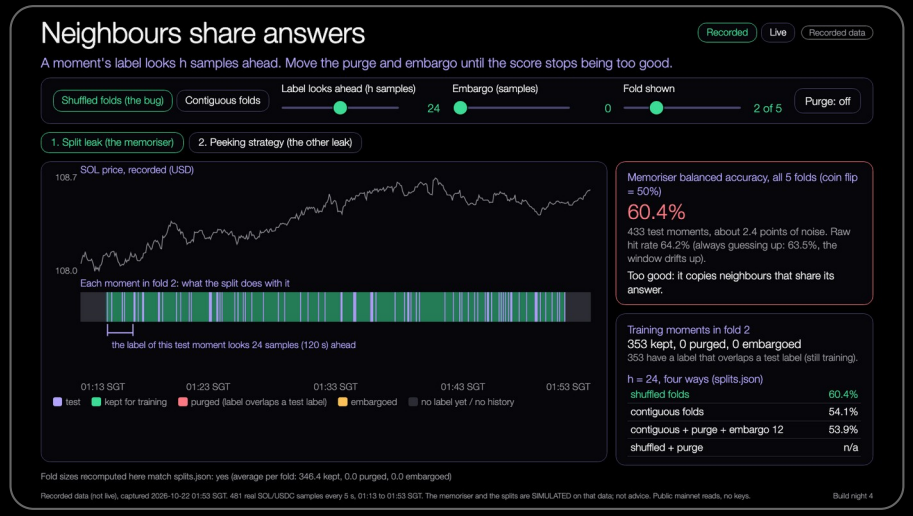
Does a memoriser score higher with shuffled folds or with a purge, and by how many points?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- accuracy for shuffled and contiguous folds
- training moments purged and embargoed

Fallback: Recorded data is the default. No network needed.




Learning Lab demo, runs from disk, recorded data.

`Try it: https://nus-solana-demos.vercel.app/0007/01-split-leakage.html`

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data). Simulated, on recorded data; not advice.

# Demo: many strategies

PREDICT FIRST

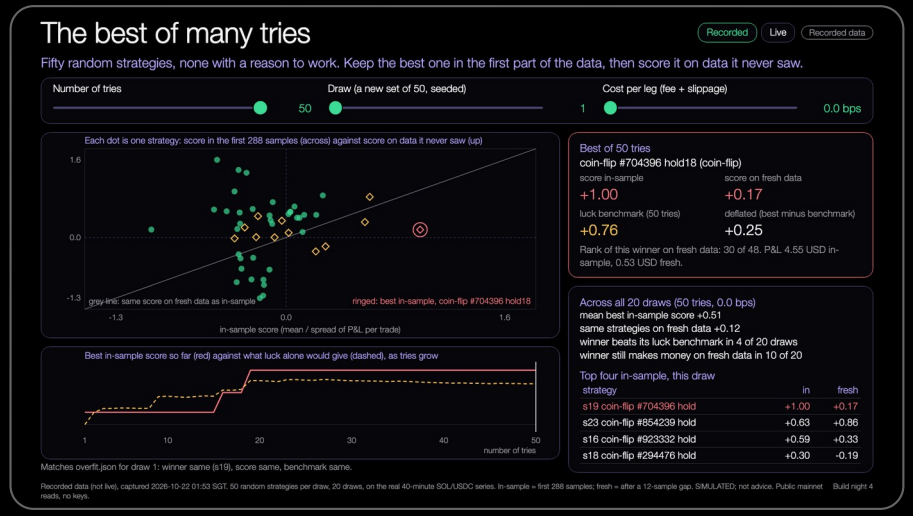
How much of the winner's score survives on data it never saw?

```
$ npm run demo -- --recorded --seeds 20
```

YOU SHOULD SEE

- the best in-sample dot, and where it lands out-of-sample
- the luck benchmark rising with the number of tries

Fallback: Same recorded data. The 50 strategies come from a seed.




Learning Lab demo, runs from disk, recorded data.

Try it: https://nus-solana-demos.vercel.app/0007/02-many-strategies.html

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data). Simulated, on recorded data; not advice.

# Demo: failure injector

PREDICT FIRST

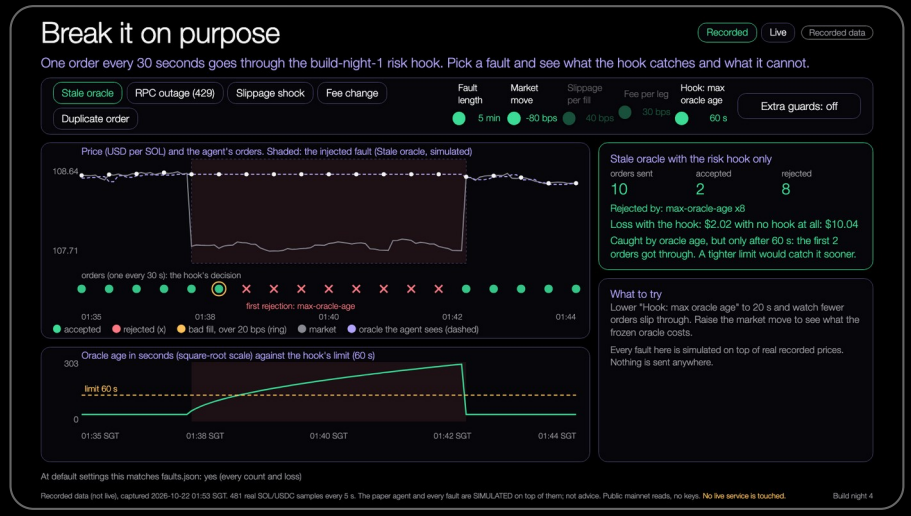
Which of the five faults does the risk hook stop?

```
$ npm run demo -- --recorded
```

YOU SHOULD SEE

- each order accepted or rejected, with the rule
- loss with the hook and with no hook

Fallback: Same recorded data. The faults are simulated on top of it.




Learning Lab demo, runs from disk, recorded data.

Try it: https://nus-solana-demos.vercel.app/0007/03-failure-injector.html

Live mode: switch it on to watch real Solana data.

Source: demo-repo/lab/screenshots (captured from the Learning Lab demos on recorded data). Simulated, on recorded data; not advice.

# Try the demos yourself

DEMOS

`nus-solana-demos.vercel.app/0007/`

Open on your laptop. Move the sliders.

No install. The same files are in the repo and open from disk.



Scan to open the demos

Source: demo-repo/lab (the four Learning Lab demos, deployed as static pages, recorded data)

# Your mission

18:40 to 19:00. Twenty minutes.

1

## Run

The sample leaky strategy

2

## Find

The planted leak with the causality test

3

## Fix

The leak, and score with a purged split

4

## Chart

The result, before and after

Finish line: one chart with a before and an after.

# Build menu

19:00 to 19:50. Pick one, or bring your own project.

| | |
|---|---|
| BEGINNER | Audit your own project for a leak |
| STANDARD | Purged split on your strategy |
| STANDARD | A feed-health alert |
| ADVANCED | A regression test for a failure you injected |

# Three takeaways

1. Neighbouring moments leak: cut folds by time, then purge and embargo.

2. The best of many tries is mostly luck: deflate it and test on fresh data.

3. Inject failures: the hook stopped the stale data but not slippage, fees or duplicates.

# Branch out

After tonight, in your own time.

BEGINNER

Run the causality test on a signal from your own project.

STANDARD

Raise the number of tries from 10 to 100 and chart the best score against the luck benchmark.

ADVANCED

Add a guard for a fault the hook missed, with a regression test that fails first.

# Share back

Before you leave, and by next Thursday.

TONIGHT

1

Push your branch to your own repo.

THU 4 FEB

2

Bring one honest number to Workshop 4.

ANY TIME

3

Send your repo link to the leads.

# Get the code

REPO

`github.com/travisim/solana-agentic-trading-nus`

`folder: 0007-backtest-validation-failure-testing/demo`

```
1  $ npm install
```

```
2  $ npm run demo -- --recorded
```

Run both from the demo/code folder. Node 24. No keys, no AI credits. Recorded data is bundled.

Scan for this session's code

# Let's connect



LinkedIn

linkedin.com/in/travisim



Telegram

t.me/travisim


SOLANA